

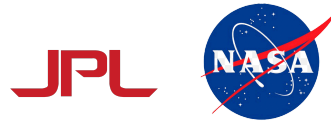


College of Engineering & Applied Science
UNIVERSITY OF COLORADO **BOULDER**

ECEN4620 Final Report

Mars Hard Lander Avionics

WATTS UP - JPL



Names: Abdulrahman Almutairi, Ahmad Sai, Anthony Hamamji, Conner Neuhart,
Nasser Al Muaili, Bilim Sydykov

April 27th, 2026

Elevator Pitch:

Have you ever heard of a Mars hard lander? A Mars hard lander is a low-cost alternative to traditional landing systems that eliminates complex landing mechanisms by surviving a direct high-impact landing. Our team developed the avionics system that enables environmental data collection, onboard storage, and reliable wireless transmission to a Mars relay network. The system is powered by a combination of solar energy and a Li-ion battery pack, allowing for sustained operation in harsh environments. Additionally, our design supports modular payloads, making it adaptable for a wide range of future Mars missions.

Executive Summary:

The Mars Hard Lander Avionics System is a low-cost, resilient electronics platform designed to operate in extreme martian conditions. Traditional landing systems rely on complex mechanical deceleration mechanisms, which significantly increase cost, risk, and system complexity. This project enables a hard landing approach by designing an avionics system capable of surviving impact, collecting environmental data, storing data onboard, and reliably transmitting that data after landing. This approach reduces overall system complexity and cost while improving mission robustness in harsh environments. The system incorporates a sustainable energy solution using solar panels and a Li-ion battery, enabling continuous operation for more than two Martian sols (~49 hours). Additionally, the design supports modular payloads, making it adaptable for a wide range of future Mars missions.



Figure 1: Fully integrated Mars Hard Lander avionics system enclosed within a protective housing for structural and thermal resilience.

At a high level, the system operates by periodically collecting environmental and system health data, storing it locally, and transmitting it during scheduled communication windows. The system is built around a dual-microcontroller architecture that balances performance and power

efficiency. A low-power STM32L4 microcontroller acts as a scheduler, periodically waking a high-performance STM32H7 microcontroller to perform data collection, storage, and communication tasks. This approach minimizes the energy consumption while maintaining system functionality over extended periods of operation.

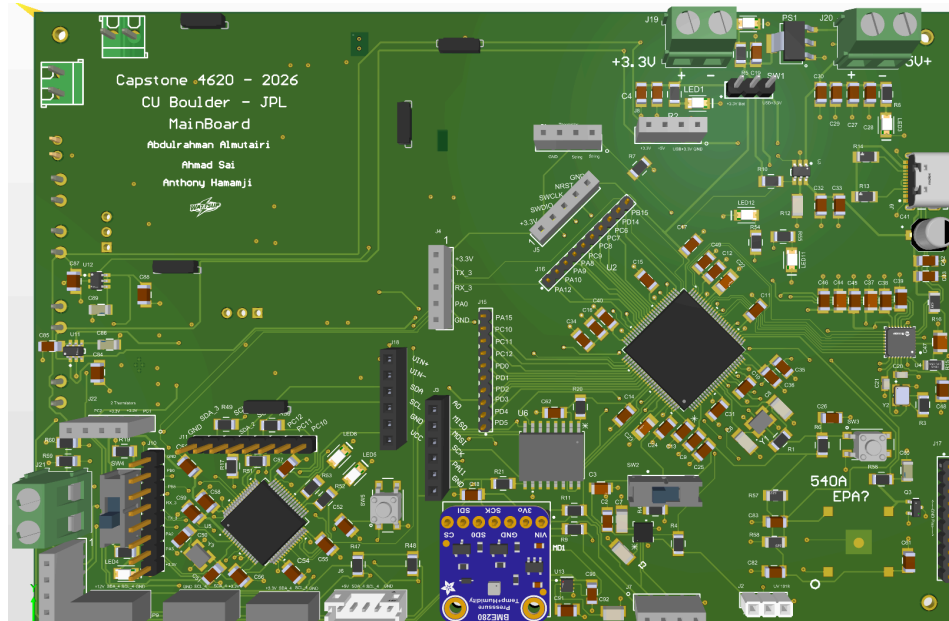
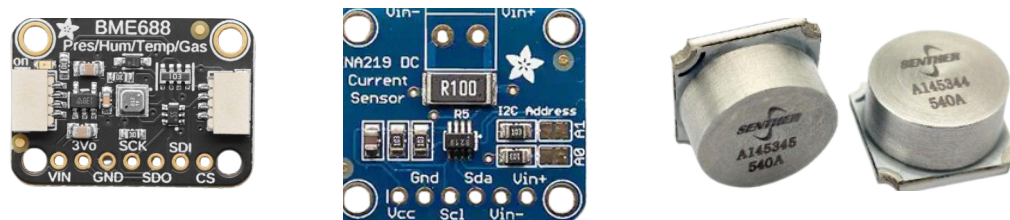


Figure 2: Front layout of the main avionics PCB showing microcontrollers, sensor interfaces, and communication circuitry.

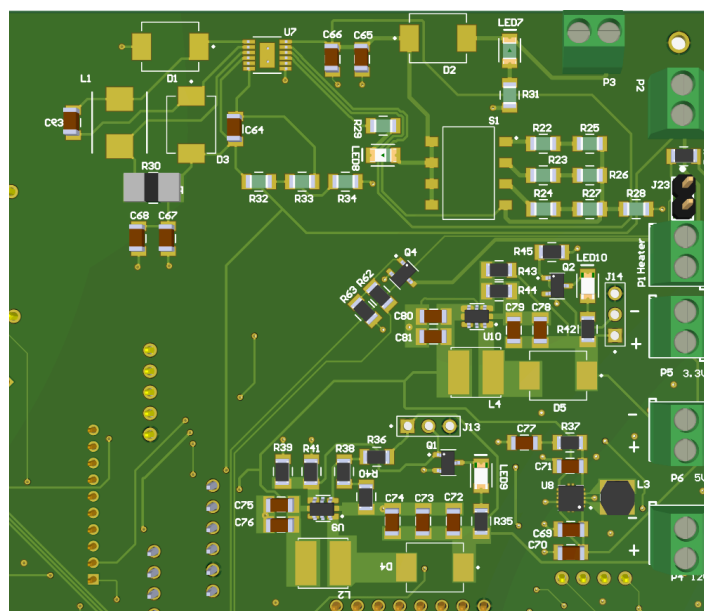
The system performs four primary functions: environmental sensing, onboard data storage, wireless transmission, and modular payload support. Environmental data is collected using onboard sensors including the BME688 for internal temperature, humidity, and pressure, along with a voltage monitoring sensor (INA219) and thermistors to provide battery voltage, current, and temperature measurements for system health monitoring. Thermistor measurements are also used to control a heater circuit, enabling the system to maintain the battery within a safe operating temperature range in extreme environments. Data is stored locally in SPI flash memory and later transmitted through a PlutoSDR module. The STM32H7 utilizes a high-speed USB interface (ULPI) with an external USB3300 PHY to transfer stored data from memory through the H7 to the PlutoSDR for wireless transmission.



The system is powered by a dedicated power distribution subsystem located on the rear of the main board. This subsystem is designed to operate from a 1S9P battery pack, with V_battery ranging from approximately 2.7 V to 4.2 V. From this battery voltage, the system generates three regulated power rails: a 12 V rail using a boost converter, a 5 V rail using a boost converter, and a 3.3 V rail using a buck-boost converter.

The subsystem also includes a solar charging circuit that allows the battery to be charged at up to 2 A. This enables the system to operate from the battery while simultaneously charging it from the solar panel. In addition, the solar charging circuit provides dual power input capability, allowing the system to draw power from both the solar panel and the battery.

A battery heater circuit is also included in the power distribution subsystem. This circuit is controlled by a digital output pin connected to the gate of a MOSFET. The MOSFET acts as a switch, allowing up to 3 A of current to flow through a copper heating wire when heating is required. All of these circuit elements are shown in Figure 4.



Testing demonstrated successful environmental data acquisition, onboard storage, and wired data transmission under controlled conditions. Due to regulatory restrictions on operating in the 430 MHz frequency band, over-the-air RF transmission was not performed during testing. Instead, a wired connection was used to validate the end-to-end communication pipeline. The system exceeded the required power performance, operating for over two Martian sols, with testing demonstrating continuous operation exceeding 100 hours under load. Core functionalities such as sensor integration, low-power scheduling, and data storage were successfully implemented. However, limitations remain in RF transmission validation, particularly for extended communication cycles. Despite these constraints, the system successfully validated the overall architecture and this work demonstrates that a low-cost, modular avionics system can reliably operate in extreme environments and serves as a strong foundation for future planetary exploration platforms.



Figure 5: PlutoSDR Receiver Dashboard (Received Sensor Data)

Requirements:

Use cases

Autonomous Environmental Monitoring

- Upon deployment, the system initializes automatically and enters an operational cycle without requiring user interaction.
- The STM32L4 microcontroller periodically wakes the STM32H7 to collect environmental data including temperature, pressure, humidity, UV, and acceleration.
- Sensor data is sampled at defined intervals and stored locally in onboard memory with appropriate formatting for later retrieval.
- The system maintains measurement accuracy within specified ranges while operating under extreme environmental conditions.
- From the customer's perspective, the lander functions as a fully autonomous data collection platform, requiring no manual control after deployment.

Low-Power Survival Operation

- The system is designed to maximize energy efficiency through a hierarchical control architecture.
- The STM32L4 microcontroller remains active in a low-power state and manages system scheduling, while the STM32H7 is only activated when high-performance tasks are required.
- During idle periods, the STM32H7 and other high-power subsystems are powered down to conserve energy.
- This duty-cycled operation allows the system to operate for extended durations (greater than two Martian sols) under limited energy availability.
- From the customer's perspective, the system provides long-term reliability and sustained operation without maintenance, even in unpredictable and resource-constrained environments.

Data Storage and Transmission Cycle

- The system continuously buffers collected sensor data in onboard SPI flash memory to ensure no data is lost between communication windows.
- During scheduled transmission intervals, the STM32L4 activates the STM32H7 and communication subsystem.
- Stored data is transferred from onboard memory through the STM32H7 and sent to the PlutoSDR via a high-speed USB interface (ULPI with USB3300 PHY).

- Data is transmitted within defined communication windows (e.g., 5 minutes) at or above the minimum required data rate.
- Due to regulatory constraints, system validation was performed using a wired interface; however, the architecture supports RF transmission for future deployment.
- From the user's perspective, this creates a predictable and reliable data retrieval cycle with minimal interaction required.

Marketing Requirements (MRD):

- Solar-power system with battery storage capability
- Autonomous environmental data collection
- Measurement of:
 - Temperature (internal and external)
 - Humidity
 - UV light
 - Acceleration
- Data transmission to a receiver
- Autonomous operation without human intervention
- Low power survival state control by L4 managing H7

Engineering Requirements (ERD):

- **Thermal survival thresholds:**
 - External minimum (-40 C)
 - Internal minimum (-20 C)
- **Environmental sensing within defined ranges and accuracies:**
 - External temperature: -75 C to 80 C (with an accuracy of ± 3 C)
 - Internal temperature (spot): -35 C to 35 C (with an accuracy of ± 3 C)
 - Internal temperature (ambient): -35 C to 35 C (with an accuracy of ± 2 C)
- **Humidity:**
 - 20-50% RH (± 4)
- **Acceleration:**
 - 0-250G (± 5 G)
- **UV measurement:**
 - 240-370 nm ($\pm 10\%$)
- **Battery life without charging:**
 - 49.32 hours
- **Voltage measurement:**

- 0 to 4.2V $\pm$ 0.05V
- **Data handling:**
 - Max storage period: 3 months
 - Storage capacity: 1Gbit
 - Minimum datarate: 150kpbs
- **Payloads:**
 - 3.3V, 5V, and 12V payload support

Stretch Goals and Achieved Requirements:

- **Thermal Performance margins:**
 - Our testing limitations prevent full validation at the lower temperature extremes. The available freezer could only reach about -40C, which is above the minimum external requirement
- **System longevity (2-year target):**
 - The system was designed for long-term operation, but this lifespan could not be validated within the project timeframe.
- **Battery life target (49.32 hours):**
 - We achieved consistent battery life and hit our target.
- **Autonomous operation:**
 - Human intervention was still required during operation, meaning the system did not fully act in an autonomous fashion.
- **Solar power system:**
 - The solar power component did not function as intended.
- **Data transmission:**
 - Data transmission was successfully achieved, though maintaining optimal performance under all conditions will still require improvements.

Technology Options:

Microcontrollers

Most central to our technology options was deciding what components we would use for our mainboard PCB. With a focus on intermittent high performance tasks and overall low power consumption, we settled on a dual microcontroller architecture that allows us to have always on monitoring with available high performance capabilities when needed for data collection or transmission.

The first microcontroller we selected was our high performance processor, the STM32H7. The up to 550Mhz clock speed of our H7 processor allows us to complete all of our required tasks within our time limits and be able to support our high speed USB connection for data transmission to the PlutoSDR for RF communication. Alongside this we selected the STM32L4 for its extremely low power usage. The L4 allows us to monitor our battery voltage, control our internal heater, as well as keep track of system time.

Knowing what we know now, we feel pretty satisfied with our architecture decisions here. Given our experience level and time constraints this dual microcontroller architecture performed relatively well and was able to be utilized to implement the main functionality of our system. Given more time, we could have considered a more redundancy focused approach that had multiple copies of the same microcontroller that could take over tasks and management if there was to be some type of failure. This approach would have built in redundancy and error correction that would much surpass what we currently have, but we would have to weigh these benefits against the implementation costs (both budget costs and development time).

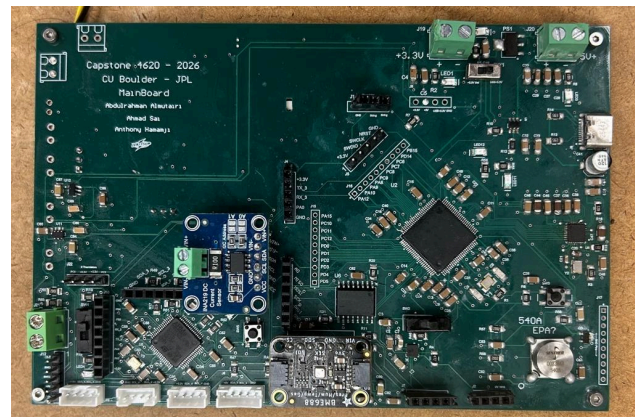


Figure 6: Fully Populated Hard Lander Avionics Mainboard

USB Transfer Software

We used the open-source TinyUSB software library on our high power microcontroller to facilitate high speed USB transfers from our NAND storage to our PlutoSDR board. This allows us to transmit megabytes of sensor data and diagnostic information to the RAM on the pluto where it is then encoded into our modulation and custom packeting scheme. This library has out of the box support for CDC-ACM communication with the STM as host, which allows us to



Figure 7: TinyUSB Logo

execute commands on the embedded linux device. But, we also implemented a bulk vendor endpoint that allows us to directly transfer large amounts of data directly into the memory on the board.

Radio Subsystem

The radio board we selected was the PlutoSDR Nano, a third-party PlutoSDR board which allows for RF communication at 430Mhz with our RRC Filtered BPSK modulation. This specific board uses the AD9363 transceiver and a Zynq7010 FPGA/MCU, allowing for onboard processing from packetization to applying our filtering and BPSK modulation. This board also has a USB interface which allows us to connect to our TinyUSB implementation and transfer sensor data to the board's memory in anticipation of an RF transmission.

The PlutoSDR itself was very fun to use, it is able to output and receive a wide range of signals and modulations and was well within our budget. The fact that the pluto runs an ARM architecture with embedded linux was also interesting because we had to cross-compile our programs and transfer files and configure the pluto using tools like ssh. Understanding the built-in software on the pluto was critical for getting the communication systems to work, and utilizing the built in buffers and FPGA connections allow us to transfer data directly from RAM into the AD9363 transceiver to output our RF signal.

This specific board was a bit of a hassle to use in some situations, particularly because the construction quality itself wasn't the best. This led to some reliability issues and erroneous behavior that we suspect are a result of poor solder connections on the Zynq7010 (reception difficulties when running the board for a long time and the chip heats up, booting inconsistencies that sometimes lead to a lack of USB enumeration, etc).

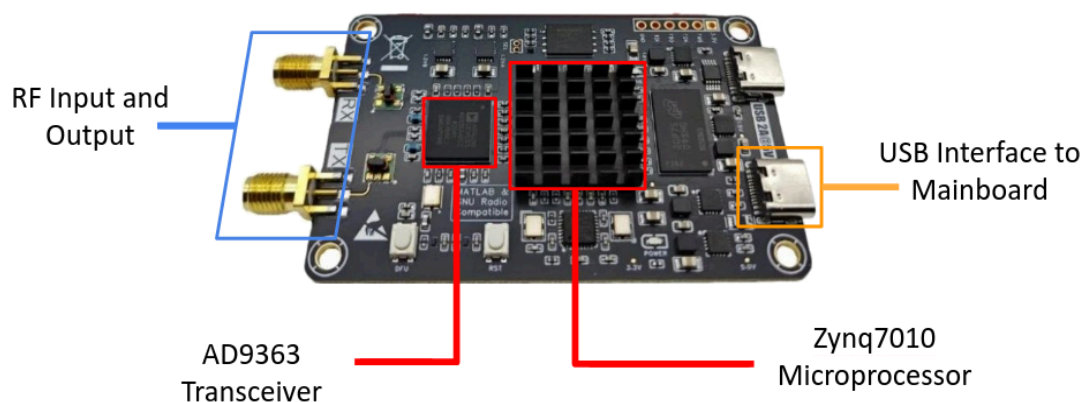


Figure 8: Software Defined Radio PCB (PlutoSDR)

Choosing this board did have some benefits, we were able to properly create the underlying communication software and send transmissions from our system to our receiver at our desired frequency and modulation. The processing power of the Zynq7010 met our needs and allowed us to almost entirely perform RF processing on the PlutoSDR itself rather than on our STM32H7. But, the reliability issues as well as complexities caused by the USB interface could ideally have been avoided if we were able to integrate a custom RF PCB into our design. Knowing what we know now we would have decided to pursue a custom implementation and used a quadrature modulator based setup that would have cut out the need for the embedded linux software on the pluto and avoided some of the aforementioned problems.

Storage

Another component we had to think about was our storage chip, which would store all of our collected sensor data for months at a time. We settled on the IS37SML01G1-MLI-TY, which is a 1Gbit NAND module. This capacity allows us to run our system for long periods of time without losing data before a transmission is available. The 104Mhz clock speed of our memory also allows for our high speed USB transfer to take place, as the transfer starts at the NAND memory, transfers to the RAM of the STM32H7, and then is packaged into a USB transfer into the RAM of the PlutoSDR. This prevents the storage from becoming a bottleneck for the high speed transfer and ensures that we don't miss our 5 minute transmission window.

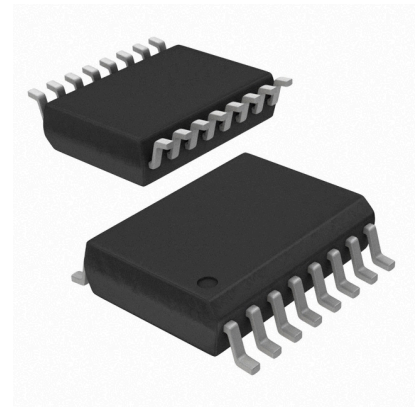


Figure 9: IS37SML01G1-MLI-TY 1Gbit NAND Memory Chip

GNURadio

GNURadio is an RF software toolkit that allowed us to develop our modulation and packeting scheme for our RF communication. We were able to integrate our PlutoSDR receiver for live demodulation and data reception, as well signal visualization through their display tools. We also used the modulation flow as a base for a more optimized version of the transmission code written in C that we eventually put into the transmitter pluto which was used inside the hard lander enclosure. GNURadio was the perfect tool for us to use considering its built-in integration with the PlutoSDR and its python-based block flow graphs for modulation, demodulation, encoding, and decoding. It was a great tool that we haven't thought twice about using, and for any changes we would have made in technology selection, choosing to use software other than GNURadio would not have been one of them.



Figure 10: GNURadio Logo

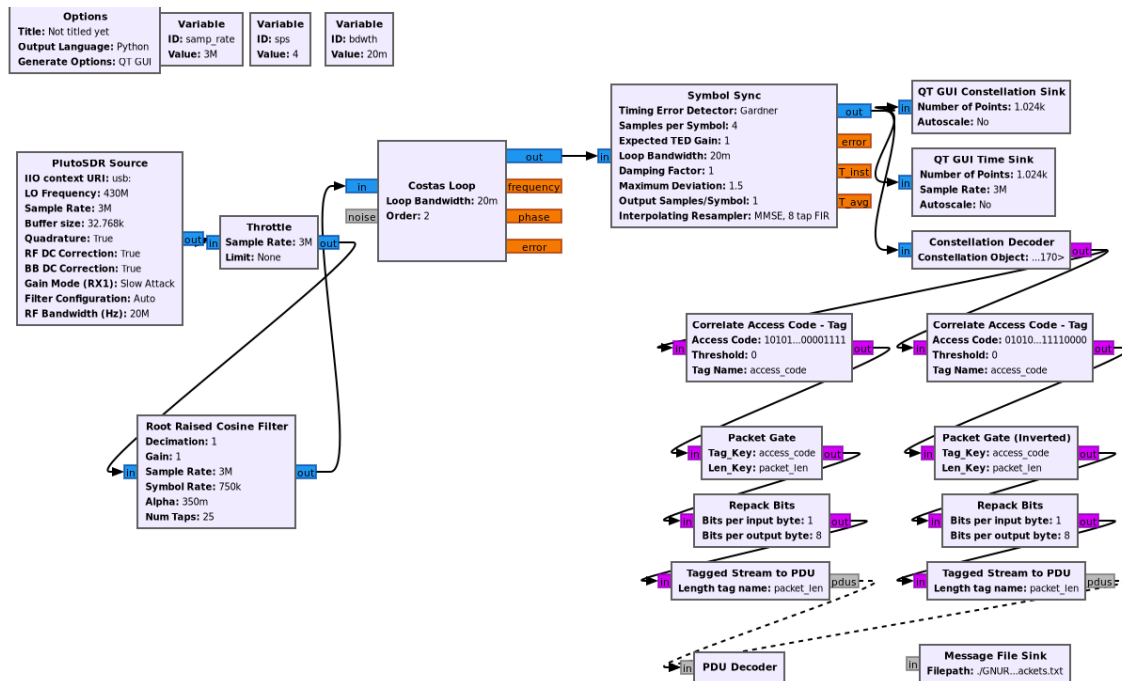


Figure 11: GNURadio BPSK demodulation and decoding flow for PlutoSDR receiver

Sensors

The sensor suite was designed to provide both environmental data and system health monitoring. The BME688 measures temperature, humidity, and pressure through a single I2C interface, reducing system complexity.

System health is monitored using an INA219 current and voltage sensor along with thermistors placed on the battery. These measurements track power usage and battery temperature, with thermistor data used to control a heater circuit for safe operation.

A 540A piezoelectric accelerometer is used to detect high-g impact events during landing, while the IAM-20380 gyroscope measures angular velocity to capture rotational motion and system orientation during descent and landing.

Power

The power subsystem uses several technology options to support battery charging, voltage regulation, and thermal control. The LT3652EMSE battery charging IC was selected because it provides a straightforward solar charging solution that has been proven to work reliably in many similar projects. This IC is responsible for controlling the battery charging process based on the input power available from the solar panel.

For the 3.3 V rail, the TPS63001DRCCR buck-boost voltage regulator was chosen. A buck-boost regulator was necessary because the battery voltage can range from 2.7 V to 4.2 V, meaning it may fall below the required 3.3 V output. Without a buck-boost converter, the system could lose the 3.3 V rail when the battery voltage drops below 3.3 V.

The system also uses MT3608 boost converters to generate the 5 V and 12 V rails. These converters were chosen because they are reliable, easy to obtain, and simple to integrate into the design. In addition to the voltage regulation circuitry, the system includes a battery heater made from 32 ft of 26 AWG copper wire with a resistance of approximately 1.3 Ω . Copper wire was selected because it is inexpensive and simple to implement as a resistive heating element. This specific wire length was chosen to achieve a low enough resistance to draw sufficient current from the battery, allowing the wire to generate enough heat to meaningfully warm the battery.

How:

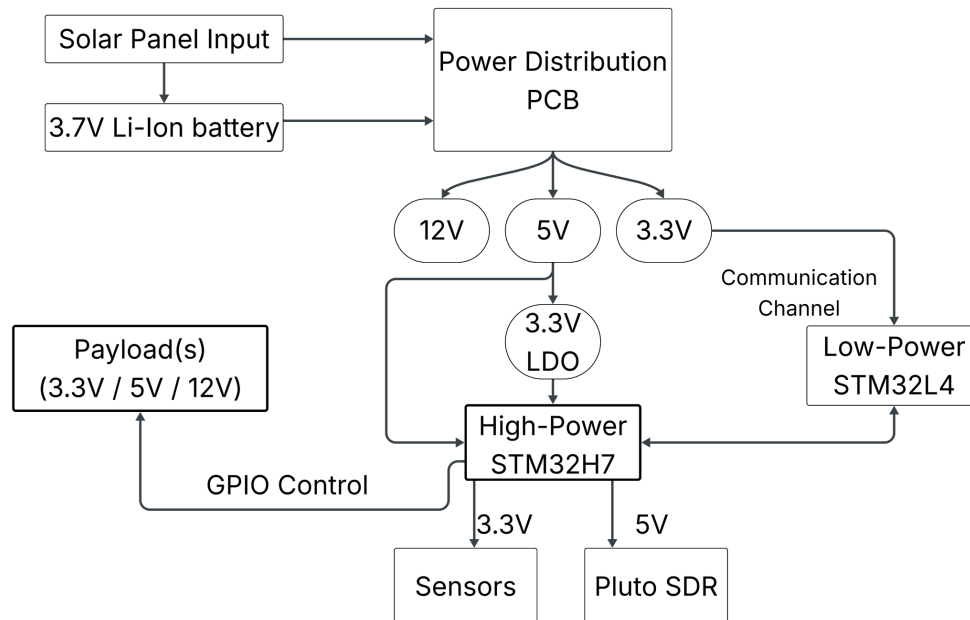


Figure 12: System-level function block diagram illustrating power distribution, control flow, and data paths between the STM32L4, STM32H7, sensors, storage, and communication subsystems.

The system is organized around a power distribution PCB, a dual-microcontroller control architecture, a sensor payload, onboard storage, and a radio communication path. Power is supplied through both a solar panel input and a 3.7 V lithium-ion battery pack. These inputs feed into the power distribution PCB, which generates the required system voltage rails, including 12 V, 5 V, and 3.3 V. These rails are then distributed to the payloads, sensors, microcontrollers, and radio hardware depending on their voltage requirements.

The control system uses two STM32 microcontrollers with separate roles. The low-power STM32L4 acts as the scheduler and power-management controller. It monitors the battery status through the voltage sensor and is responsible for maintaining low-power operation when the system does not need to be fully active. It also controls the battery heater, allowing the system to regulate the battery temperature when needed. The STM32L4 communicates with the high-power STM32H7 through a UART communication channel, sending commands that determine when the higher-power processor should wake up or perform specific tasks.

The STM32H7 serves as the main processing unit for the system. It collects data from the sensor suite, which includes temperature, pressure, humidity, UV, and current sensors. After collecting the sensor data, the STM32H7 can store the information in onboard storage for later

use. When communication is required, the STM32H7 packages the collected data and sends it to the Pluto SDR. The Pluto SDR then transmits the data through the antenna.

The power architecture also supports the different voltage requirements of each subsystem. The sensors are powered from the 3.3 V rail, while the Pluto SDR is powered from the 5 V rail. The high-power STM32H7 receives regulated power through the 5 V and 3.3 V regulation path, including a 3.3 V LDO where needed. The payloads can be powered from the available 3.3 V, 5 V, or 12 V rails and are controlled through GPIO signals from the STM32H7. This allows the processor to enable or disable specific payloads depending on the mission state.

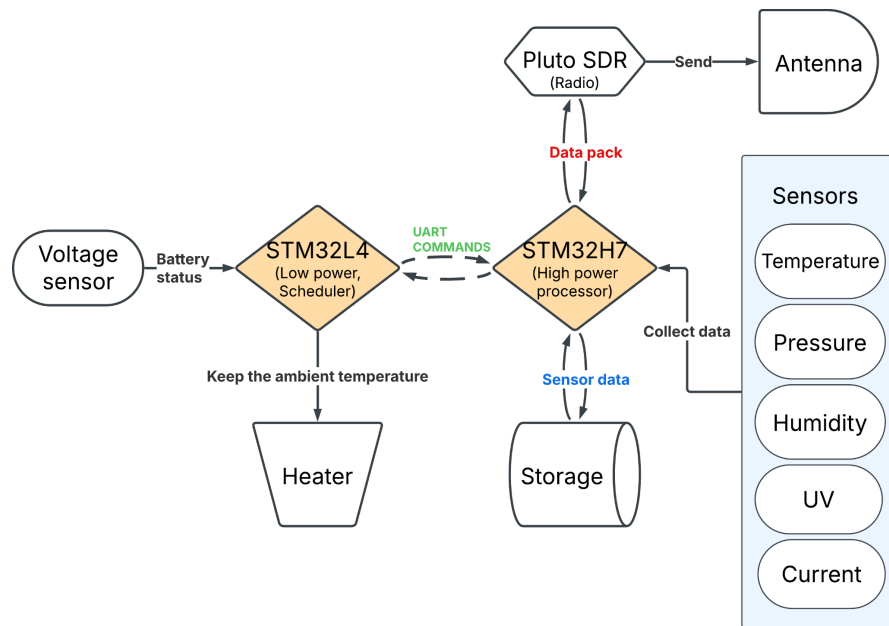


Figure 13: System operation diagram illustrating control and data flow between the STM32L4 scheduler, STM32H7 processor, sensors, storage, and communication subsystem.

The system is enclosed within a multi-layer protective housing designed to provide both mechanical durability and thermal insulation. The enclosure uses a combination of PLA and TPU materials for structural strength and impact resistance, while internal layers of polyethylene foam and external mylar wrapping reduce heat loss. This design helps protect sensitive electronics and the battery from extreme temperatures and high-g forces experienced during landing, supporting reliable long-duration operation in harsh environments.

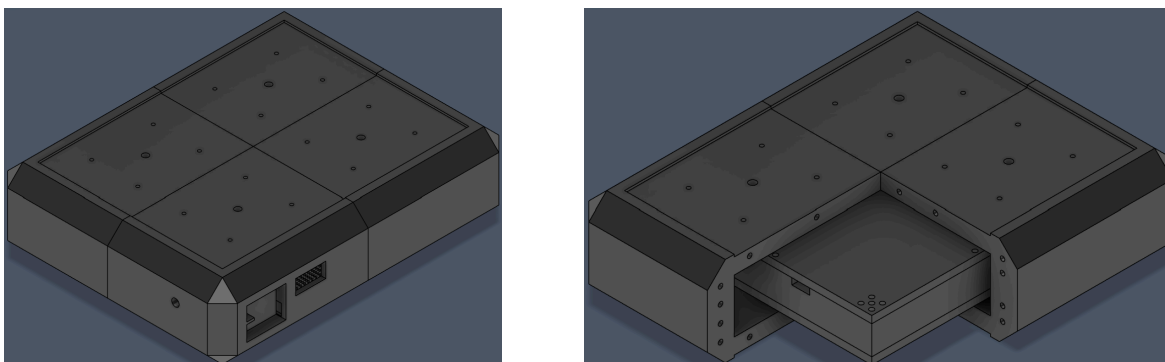


Figure 14: Final enclosure design showing the integrated housing for the avionics system, providing structural protection and thermal insulation.

Overall, the system is designed to balance low-power operation with high-performance data collection and communication. The STM32L4 remains active as a low-power supervisory controller, monitoring battery conditions and managing heater operation, while the STM32H7 is used when higher processing capability is required. The solar input and battery pack provide flexible power sources, while the power distribution PCB ensures that each subsystem receives the correct operating voltage. This architecture allows the system to collect environmental and electrical data, store it locally, and transmit it wirelessly through the SDR and antenna.

Analysis:

We were able to meet several of the core MRD and ERD requirements, especially the autonomous operation, data collection and system integration. However, there were still some limitations in the power system and RF design that prevented full completion of the requirements.

One of the biggest successes of the system was the dual MCU architecture. The STM32L4 consistently acted as the low power controller and successfully scheduled system operation. It was able to wake the STM32H7, send commands, and return system back to a low power state (sleep mode). Data collection and storage system also worked as expected. The H7 was able to collect sensor data, store it in memory and later send it for transmission.

During testing, we were able to complete bulk transfer through the PlutoSDR, which confirmed that the communication pipeline from storage to RF transmission was functional and included BPSK demodulation. Battery, sensors and enclosure temperature testing showed strong

results. The battery lasted over 48 hours under load at -40°F, each sensor survived individual temperature tests as well. The enclosure testing showed that temperature loss was slowed significantly, which supports long-term survivability of the system



Figure 15 (Left) Showing Enclosure in Freezer for testing and Figure 16 (Right) shows the data dashboard being displayed at EXPO

Unfortunately, there were a few issues with some of the power subsystem implementation that prevented the product from meeting all the ERD requirements. The most prominent issue was solar charging. For starters, there were some circuitry issues that prevented our battery from charging reliably and for long durations of time. These issues are some that need to be addressed in the future in order to meet our mission duration marketing requirement. If we had to redesign the solar charging system in the future, we would have used a more streamlined MPPT algorithm that has been implemented in the past. For example, we could have used a perturb and observe algorithm that tracks a maximum duty cycle in order to charge the battery at a stable voltage. The power subsystem as a whole worked quite well, we had no issues with both heating our battery at a stable rate and maintaining all our voltage rails.

Another challenge was the RF design. The initial AX5043 setup didn't meet our transmission requirements and had poor signal quality. We had to switch to using the PlutoSDR, which allowed us to successfully transmit the data, but it reduced the level of integration in the final system and required some external hardware.

Overall, the system fully met the requirements related to autonomous operation, data collection, and command-based control. It partially met requirements for thermal performance and power distribution, since those worked but not in the final EXPO design. Our system didn't fully meet requirements related to long-term solar powered operation.

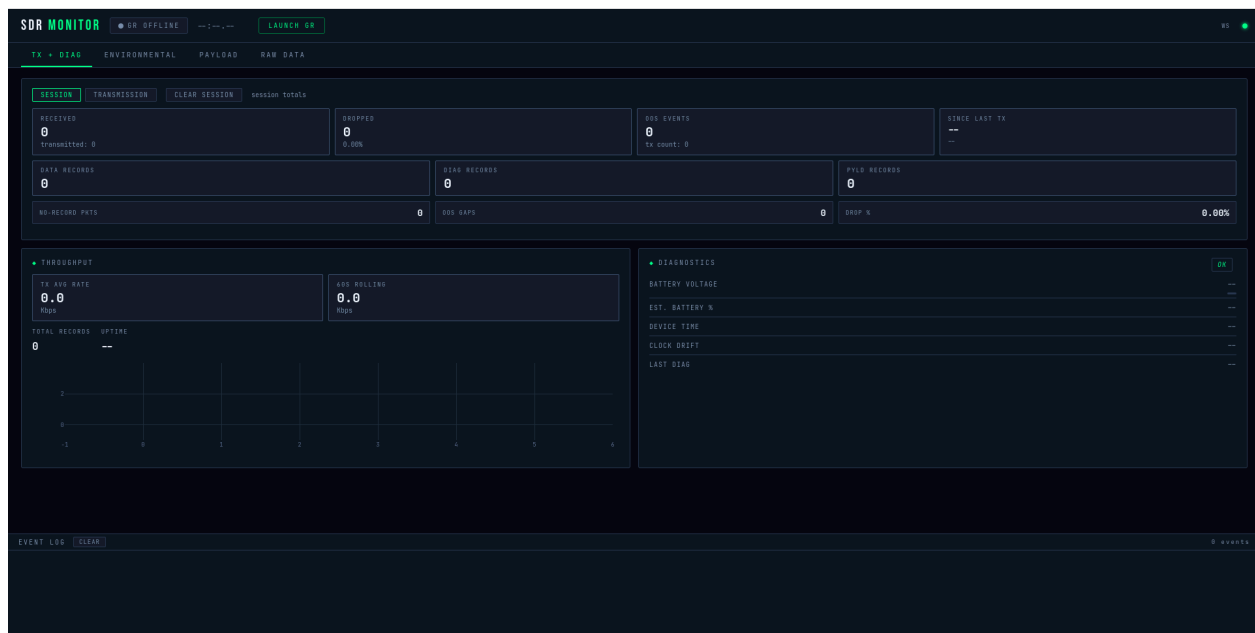


Figure 17: PlutoSDR Receiver Dashboard (Link statistics)

So What?

Through this project, we learned a lot about both system level design and real world engineering challenges. While we were able to get most of the system working, the process showed us where the biggest risks actually are.

Key technical takeaways:

- Dual MCU architecture works well for low power systems'
 - L4 scheduling + H7 execution is efficient and scalable
 - Clear separation of low power and high performance tasks
- Power systems are harder than expected
 - Custom power board design is very sensitive to layout and routing
 - Small mistakes in PCB design can cause full system failure
 - MPPT and solar charging was a “too good to be true solution”. We relied on an IC that said it could do what we needed it to but it didn’t work as reliably as we would have hoped
- RF systems require early focus
 - AX5043 failure showed that not all chips meet system requirements
 - Switching to PlutoSDR worked, but added complexity
 - RF should have been validated earlier in the project
- Integration is the hardest part
 - Individual subsystems worked well on their own
 - Combining them into one system introduced unexpected issues

System Level takeaways:

- Autonomous systems need strong testing
 - Even small bugs can break full operation cycles
 - Timing and synchronization between L4 and H7 is critical
- Power management is key for mission success
 - Turning H7 on only when needed significantly reduces power usage
 - Duty cycling is required for long term operation
- Data handling strategy worked well
 - Storing data first and transmitting later is more reliable
 - Bulk transfer is more efficient than continuous transmission

Team takeaways:

- **Abdulrahman Almutairi:**
 - Gained experience in designing a complex STM32 MCU PCB design while contributing with Ahmad on Altium 365 for version control.
 - Focused on the hardware–software junction to ensure full integration and reliable interaction between subsystems.
- **Ahmad Sai**
 - Built a strong foundation in embedded systems by working extensively with the STM32 microcontroller, including designing a custom PCB tailored to project requirements.
 - Developed and implemented software to interface with and manage a variety of sensors, improving system functionality.
- **Bilim Sydykov:**
 - Gained software designing experience with multiple MCU integration.
 - Learned more about the RF design.
 - Learned team logistics management between our engineering team and the customer (sponsor)
- **Conner Neuhart**
 - Gained valuable experience about designing and integrating an RF communication protocol for transferring data
 - Learned about MCU integration on custom PCBs. In particular, using the STM development environment and how to design software for real-time autonomously controlled operations integrated with a variety of sensors and interfaces
 - Gained experience in finance management and budget planning between each stage of the project, forecasting budgets in advance and not overspending when possible.
- **Nasser Al-Muaili**
 - Gained a deeper understanding of how 3D printed components are affected by temperature and the insulative properties of air gaps.
 - Learned more about how to use Fusion 360 and how to make efficient 3D printer compliant models.
- **Anthony Hamamji**
 - One of the main things I learned was the flow of full product development, all the way from PDR to design and integration
 - Another was hands-on experience with battery design, power distribution and various different heating element materials.

What we would do differently:

- Use a custom designed solar charging circuit instead of relying on an IC
- We would have produced one more iteration of PCBs before final integration

- Early testing would have helped us avoid redesign late in the project
- Late design changes reduced the time available for validation
- We underestimated how long RF and power debugging would take
- Early integration testing would have helped show the flaws in the designs

Our Journey

Throughout this project, our team went through multiple design iterations and learned how to work together on a complex system. At the beginning, most of our work was focused on breaking down the system into smaller subsystems such as power, RF, mainboard, and enclosure. Our brainstorming process worked well early on. We were able to explore different design options and choose components that met our requirements. However, as the project progressed, we realized that initial design decisions do not always work in practice. For example, our initial RF design using the AX5043 did not meet requirements, which forced us to redesign that part of the system later in the project.

As a team, we made most of our decisions through discussion and testing results. When something did not work, we would analyze the issue and decide whether to fix it or move to a different approach. This was especially important for the power system and RF subsystem, where we had to make changes based on real test results instead of assumptions. While individual subsystems worked well on their own, combining everything into one system introduced new issues. Timing between the L4 and H7, power stability, and communication reliability all required additional debugging.

In terms of organization, dividing the project into subsystems helped us stay on track. Each team member was responsible for a specific part, but we still needed constant communication to make sure everything worked together. Regular check-ins and meetings helped us coordinate progress and solve problems as a team.

If we were to do this project again, we would focus more on early testing and validation, especially for high-risk areas like power and RF. We would also plan for multiple PCB revisions and start system integration earlier. Overall, this project helped us understand how real engineering projects work, where design, testing, and iteration all play a critical role.